

ICT600 CORRECTED STUDENT EDITION

July 2025 Corrected Student Scheme

Moderation-aware edition for the printed 102-mark anomaly

Scope	July 2025 final examination
Evidence	Printed question, supplied scheme where available, current lecture notes, and technical verification
Status	Printed total 102; official-intended/normalised total 100

Prepared 26 September 2026

How to use this reference

Teaching-use document

Attempt the original question before revealing this scheme. This corrected edition is not an official UiTM replacement; it records known source errors and accepts equivalent technically correct answers.

- Assessment total: Printed total 102; official-intended/normalised total 100.
- Model answers show one defensible solution; alternative correct wording or code may receive full credit.
- For code, method marks remain available when the approach is sound and a minor syntax error does not invalidate the solution.
- Security improvements are shown where historical examples trusted or displayed unvalidated input.

Q1

Define four web-technology components and compare browser vs web application.

[18 marks]

Model answer

- Internet, web standards, web server, and CMS: 2 marks each.
- Five paired comparison dimensions: nature, purpose, architecture role, data handling, and scope: 2 marks each.

Marking guide: Q1(a) 8; Q1(b) 10.

Q2

Define HTML, CSS, and web development; state three good-design principles.

[12 marks]

Model answer

- Each definition: full meaning plus function (2).
- Any three principles such as consistency, navigation, responsiveness, readability, accessibility, and organised content, with explanation (2 each).

Marking guide: 6 marks per subpart.

Q3

CRUD statements and the 2025S1 enrollment query.

[18 marks]

Model answer

- INSERT, SELECT, UPDATE with WHERE, DELETE with WHERE (2 each).
- SELECT Students.Name/Major, join on StudentID, filter Enrollments.Semester='2025S1' (10).

Marking guide: Q3(a) 8; Q3(b) 10.

Q4

Framework simplification and MVC request flow.

[10 marks]

Model answer

- Any four valid benefits such as reusable code, routing, ORM/database support, security helpers, structure, or faster development (4).
- Controller receives request, Model retrieves books, Controller passes data, View renders list (2 marks each MVC role).

Marking guide: Q4(a) 4; Q4(b) 6.

Q5(a)

Identify three search-form security risks and mitigations.

[6 marks]

Model answer

- Any three valid pairs, for example SQL injection/prepared statements, XSS/contextual output encoding, data leakage/HTTPS, or information disclosure/generic errors and secure logging.

Marking guide: Risk (1) + matching mitigation (1) x3.

Q5(b)

Describe three secure-coding techniques.

[8 printed marks / 6 official-intended]

Model answer

- Parameterized queries for SQL injection (2).
- Context-aware output encoding for XSS (2).
- Password hashing, TLS, and secure session handling for authentication (2).
- If the printed 8 marks must be honored, award 2 additional marks for correctly applying the three controls to sensitive-user-data handling or for a fourth valid technique.

Marking guide: Never silently reduce the printed paper. Use 8 for historical candidate marking unless formal moderation authorizes 6; use 6 when normalising the paper to the official 100-mark design.

Correction / moderation note

The question PDF prints 8, while the official answer and generated scheme allocate 6. This makes the printed paper total 102.

Part B Q1

Validate the registration form and critically compare JavaScript with jQuery.

[15 marks]

Model answer

- Validation behavior: 8 marks; comparison/recommendation: 7 marks.
- Accept either native JavaScript or jQuery when the code and comparison are internally consistent.

Marking guide: Submit/event (2), empty checks (2), email (2), message (2); comparison and recommendation (7).

Part B Q2 - runnable Student Result Analyzer

Study the complete working exemplar below.

[See printed allocation]

Model answer

- Equivalent correct implementations that satisfy the stated behaviour should receive credit.

```
<!doctype html>
<html lang="en"><head><meta charset="utf-8"><title>Result Analyzer</title></head>
<body>
  <form id="resultForm">
    <label>Name <input id="studentName" required></label>
    <label>Five marks <input id="marks" placeholder="75,64,80,55,90" required></label>
    <button>Analyze</button>
  </form>
  <pre id="result"></pre>
  <script>
document.getElementById('resultForm').addEventListener('submit', event => {
  event.preventDefault();
  const name = document.getElementById('studentName').value.trim();
  const marks = document.getElementById('marks').value.split(',').map(Number);
  if (!name || marks.length !== 5 || marks.some(m => !Number.isFinite(m) || m < 0 || m > 100)) {
    document.getElementById('result').textContent = 'Enter a name and five marks from 0 to 100.';
  }
});
  </script>
</body>
</html>
```

```

    return;
  }
  const total = marks.reduce((sum, mark) => sum + mark, 0);
  const average = total / marks.length;
  const grade = average >= 80 ? 'A' : average >= 70 ? 'B' :
    average >= 60 ? 'C' : average >= 50 ? 'D' : 'F';
  document.getElementById('result').textContent =
    `Student: ${name}\nMarks: ${marks.join(', ')}\nTotal: ${total}\n` +
    `Average: ${average.toFixed(2)}\nGrade: ${grade}`;
  });
</script></body></html>

```

Marking guide: Apply the original question's subpart allocation and award method marks consistently.

Correction / moderation note

The official and generated examples hard-code the sample. The corrected teaching version actually accepts a name and five marks, matching the verb accept in the question.

Consistency rules

- Award equivalent technically correct syntax and wording.
- Do not reproduce a known historical error merely because it appears in a supplied scheme.
- Keep printed subpart totals fixed unless a documented moderation decision says otherwise.
- For scenario questions, connect the explanation or mitigation to the scenario.
- Record additional accepted alternatives for the next controlled revision.

July 2025 - Complete Coding Answers

Exam-schema presentation

Each answer below includes executable model code and an awardable marking schedule that sums to the printed marks. Equivalent working solutions receive the same marks when they satisfy the required behaviour.

Question 3(a) - CRUD statements

8 marks

Write complete Create, Read, Update and Delete SQL statements.

Mark allocation

Awardable element	Marks
INSERT	2
SELECT	2
UPDATE with WHERE	2
DELETE with WHERE	2
Total	8

Complete model answer

```
INSERT INTO Students (StudentID, Name, Age, Major)
VALUES (1001, 'Aina Rahman', 21, 'Computer Science');

SELECT * FROM Students;

UPDATE Students
SET Age = 22
WHERE StudentID = 1001;

DELETE FROM Students
WHERE StudentID = 1001;
```

Question 3(b) - enrollment JOIN

10 marks

Display unique student names and majors for semester 2025S1.

Mark allocation

Awardable element	Marks
SELECT DISTINCT	2
Required columns	2
Tables and INNER JOIN	2
StudentID join condition	2
Semester filter	2
Total	10

Complete model answer

```
SELECT DISTINCT s.Name, s.Major
FROM Students AS s
INNER JOIN Enrollments AS e
ON e.StudentID = s.StudentID
WHERE e.Semester = '2025S1';
```

Part B Question 1 - JavaScript validation

8 marks

Validate username and email and write the result to #message.

Mark allocation

Awardable element	Marks
Submit event and preventDefault	2
Empty-field checks	2
Email-format check	2
Relevant success/error message	2
Total	8

Complete model answer

```
const form = document.getElementById('registerForm');
const message = document.getElementById('message');

form.addEventListener('submit', event => {
  event.preventDefault();
  const username = document.getElementById('username').value.trim();
  const email = document.getElementById('email').value.trim();
  const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

  if (username === '' || email === '') {
    message.textContent = 'Username and email are required.';
  } else if (!emailPattern.test(email)) {
    message.textContent = 'Enter a valid email address.';
  } else {
    message.textContent = 'Registration Successful';
  }
});
```

Marker note

The remaining 7 marks are awarded for the written comparison: syntax/readability (2), maintenance/scalability (2), beginner suitability (1), and a justified recommendation (2).

Part B Question 2 - Student Result Analyzer

15 marks

Provide a runnable program that accepts a name and five marks, then calculates and displays total, average and grade.

Mark allocation

Awardable element	Marks
Name and five-mark input	2
Array/numeric validation	2
Total calculation	2
Average calculation	2
Correct grade boundaries	3
Complete output	2
Functions/control flow and valid structure	2
Total	15

Complete model answer

```
<!doctype html>
<html lang="en"><head><meta charset="utf-8"><title>Result Analyzer</title></head>
<body>
  <form id="resultForm">
```

```

<label>Name <input id="studentName" required></label>
<label>Five marks <input id="marks" placeholder="75,64,80,55,90" required></label>
<button>Analyze</button>
</form>
<pre id="result"></pre>
<script>
document.getElementById('resultForm').addEventListener('submit', event => {
  event.preventDefault();
  const name = document.getElementById('studentName').value.trim();
  const marks = document.getElementById('marks').value.split(',').map(Number);
  if (!name || marks.length !== 5 || marks.some(m => !Number.isFinite(m) || m < 0 || m > 100)) {
    document.getElementById('result').textContent = 'Enter a name and five marks from 0 to 100.';
    return;
  }
  const total = marks.reduce((sum, mark) => sum + mark, 0);
  const average = total / marks.length;
  const grade = average >= 80 ? 'A' : average >= 70 ? 'B' :
    average >= 60 ? 'C' : average >= 50 ? 'D' : 'F';
  document.getElementById('result').textContent =
    `Student: ${name}\nMarks: ${marks.join(', ')}\nTotal: ${total}\n` +
    `Average: ${average.toFixed(2)}\nGrade: ${grade}`;
});
</script></body></html>

```

Coding-marker consistency

- Award method marks when the core algorithm and required structure are correct.
- Do not double-penalise one syntax error that causes several later lines to fail.
- Accept equivalent DOM methods, loop forms, variable names, SQL aliases, and control structures.
- Require every behaviour named in the question; a prose description alone is not a complete coding answer.
- For unsafe historical code, award course-aligned logic while teaching prepared SQL, password hashing, validation, and output encoding.